

varmap

Fast heterogeneous key–value maps for Rust
Gavrilut Dragos

3–5×

faster than HashMap

16 B

per value, fixed cell

65 536

max keys per map

Storing mixed-type values under one map

Existing options

HashMap<K, V>

one value type only

HashMap<K, Any>

boxing + dynamic dispatch

enum-keyed maps

homogeneous values

anymap / typemap

keyed by type, not name

What we want

- ✓ Name → typed value (bool, ints, &str, &[u8], structs)
- ✓ Zero-copy reads — &str and &[u8] borrowed from arena
- ✓ Compile-time keys when possible (no runtime hashing)
- ✓ Faster than HashMap on read-heavy workloads

Three maps, one value layer

EnumVarMap<E>

key: enum variant

fastest, fixed slots

VarMap

key: compile-time hash

no runtime hashing

StrVarMap

key: runtime &str

dynamic keys

Shared value layer – 16-byte tagged ValueKind

Bool · I8...I64 · U8...U64 · F32 · F64 · SmallString(≤14B) · SmallBytes(≤14B) · ArenaIndex for strings/bytes/i128/u128/custom

Arena (Vec<u128>, append-only between clear())

Holds large strings, byte slices, i128/u128, and custom Copy types. Reads return borrowed slices — no copy.

One API, mixed value types

StrVarMap – runtime string keys

```
use varmap::StrVarMap;

let mut map = StrVarMap::new();

map.set("port", 8080u16);
map.set("enabled", true);
map.set("host", "api.example.com");

assert_eq!(map.get_u16("port"), Some(8080));
assert_eq!(map.get_bool("enabled"), Some(true));
assert_eq!(map.get_str("host"), Some("api.example.com"));
```

VarMap + var! – compile-time keys

```
use varmap::{VarMap, var!};

let mut map = VarMap::new();

// var!() hashes at compile time – no runtime FNV
map.set(var!("user.age"), 32u32);
map.set(var!("user.name"), "Alice");

let age: u32 = map.get(var!("user.age")).unwrap();
let name: &str = map.get(var!("user.name")).unwrap();
```

Same value layer — switch maps without rewriting your getters.

Closed key sets and custom types

EnumVarMap – O(1) direct indexing

```
use varmap::{EnumVarMap, EnumVarMapKey};

#[derive(EnumVarMap, Copy, Clone, Debug)]
#[repr(u16)]
enum ConfigKey { Port, Enabled, Host }

let mut m = EnumVarMap::<ConfigKey>::new();
m.set(ConfigKey::Port, 8080u16);
m.set(ConfigKey::Enabled, true);
m.set(ConfigKey::Host, "api.example.com");

// O(1) – direct array index, no hashing
let p = m.get_u16(ConfigKey::Port);
```

Custom Copy types via #[derive(VarMapValue)]

```
use varmap::{StrVarMap, VarMapValue};

#[derive(VarMapValue, Copy, Clone, Debug)]
struct Point { x: i32, y: i32 }

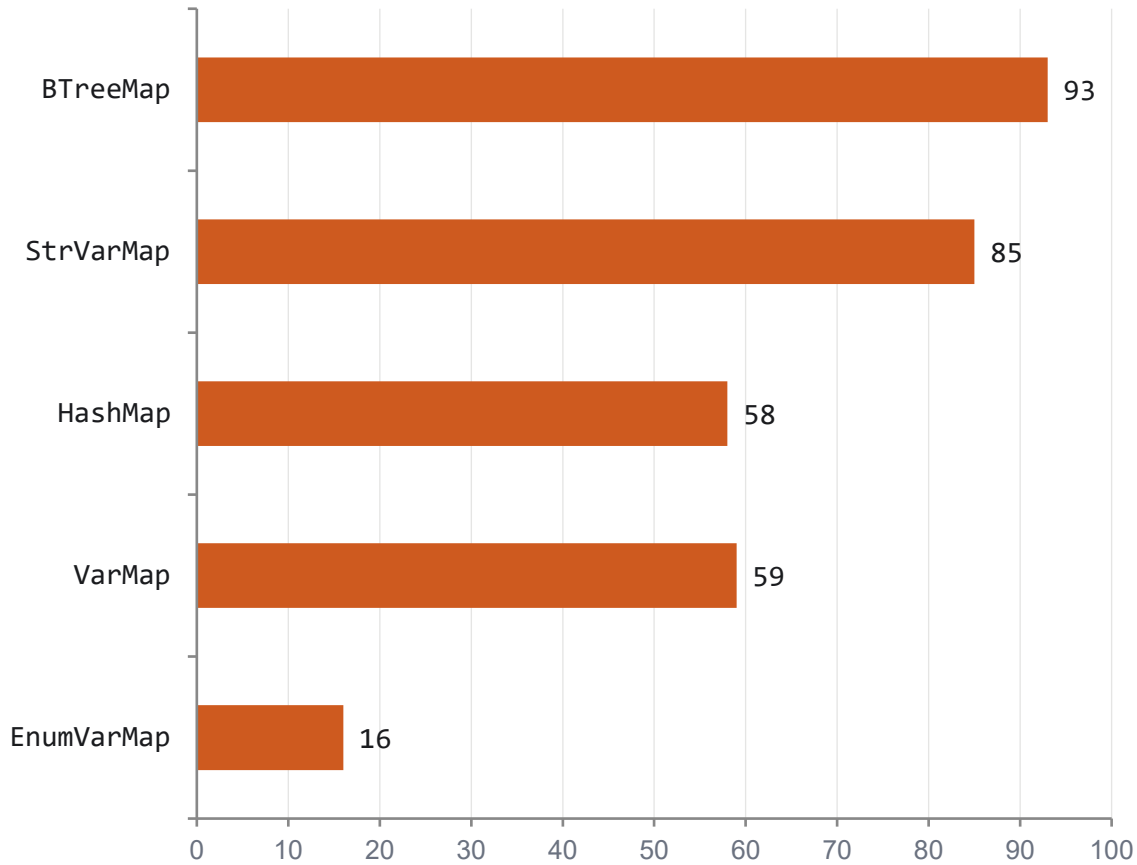
let mut m = StrVarMap::new();
m.set("origin", Point { x: 1, y: 2 });

// Zero-copy: returns &Point from the arena
let p: &Point = m.get("origin").unwrap();
assert_eq!(p.x, 1);
```

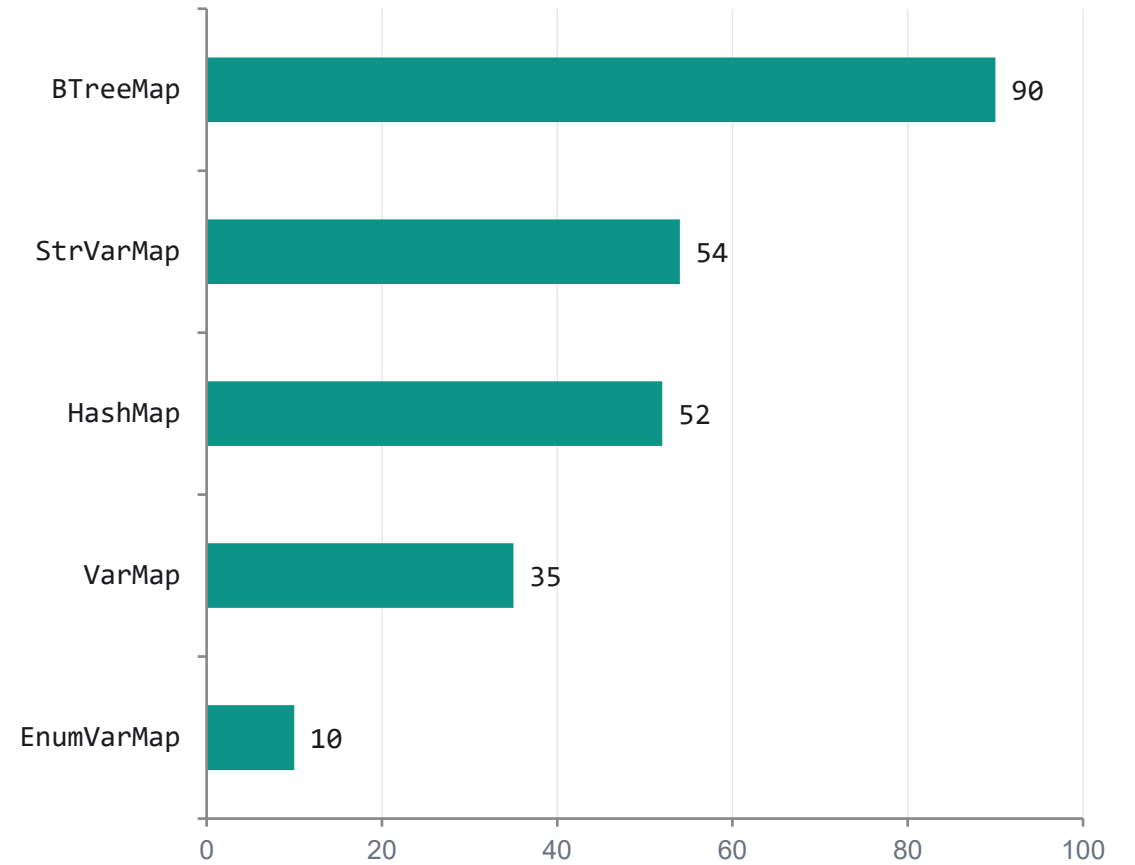
Constraints: Copy + alignment 1–16. Strings, byte slices, and i128/u128 land in the arena automatically.

Read benchmarks – the sweet spot

Read – large strings (ms, lower is better)



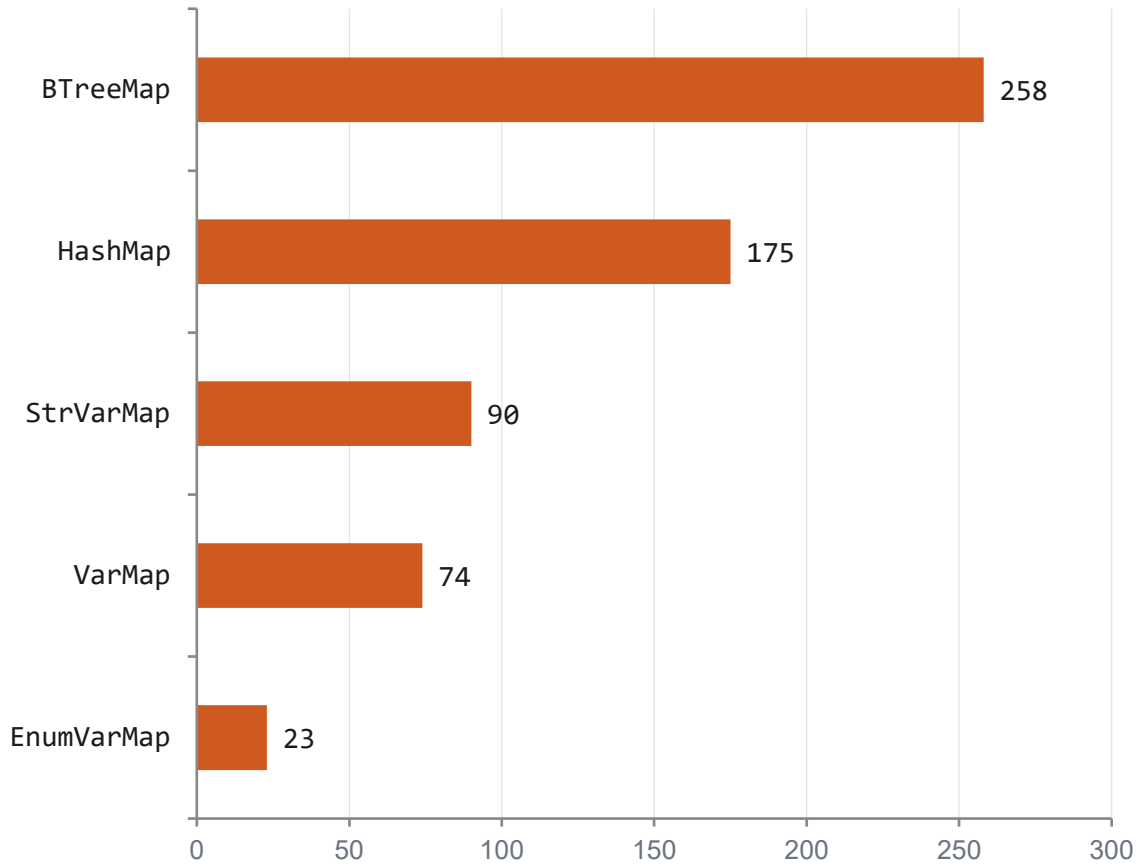
Read – small strings (ms, lower is better)



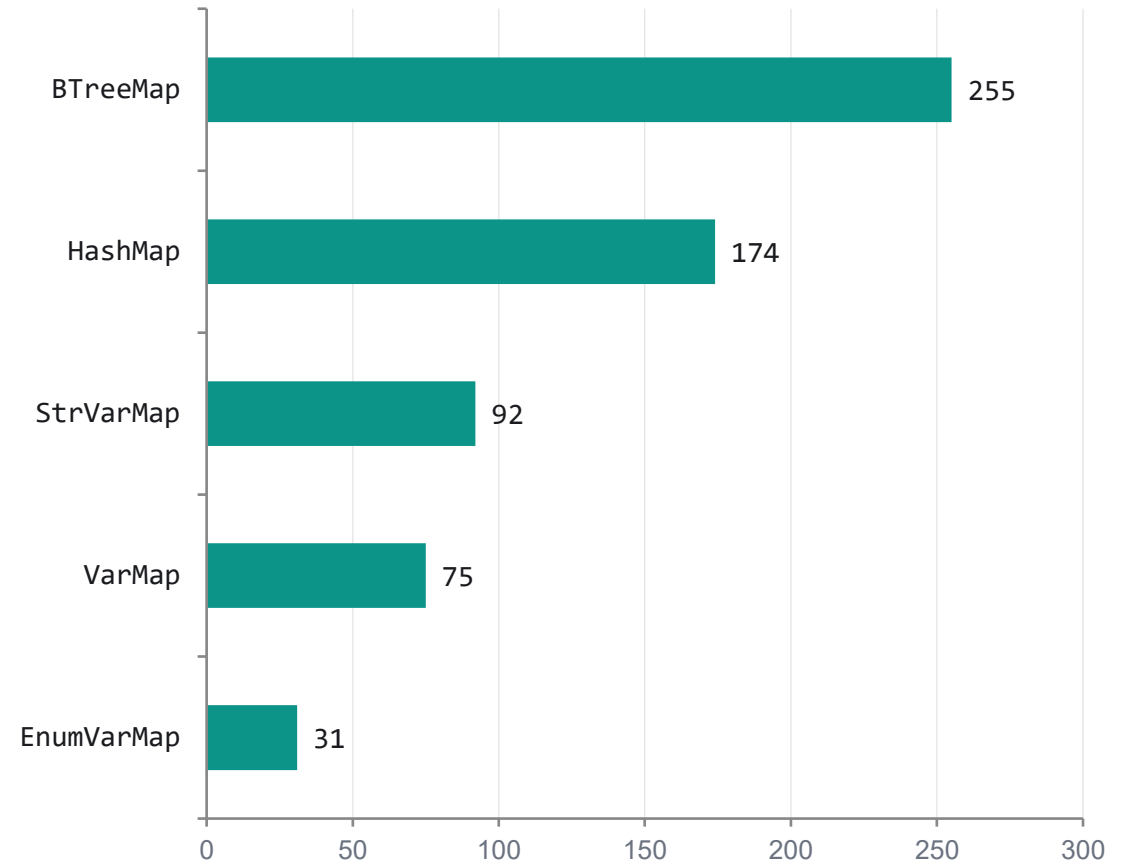
10 000 iterations × 256 keys. EnumVarMap: ~5× faster than HashMap on small-string reads.

Update benchmarks – clear & repopulate

Update – large strings (ms, lower is better)

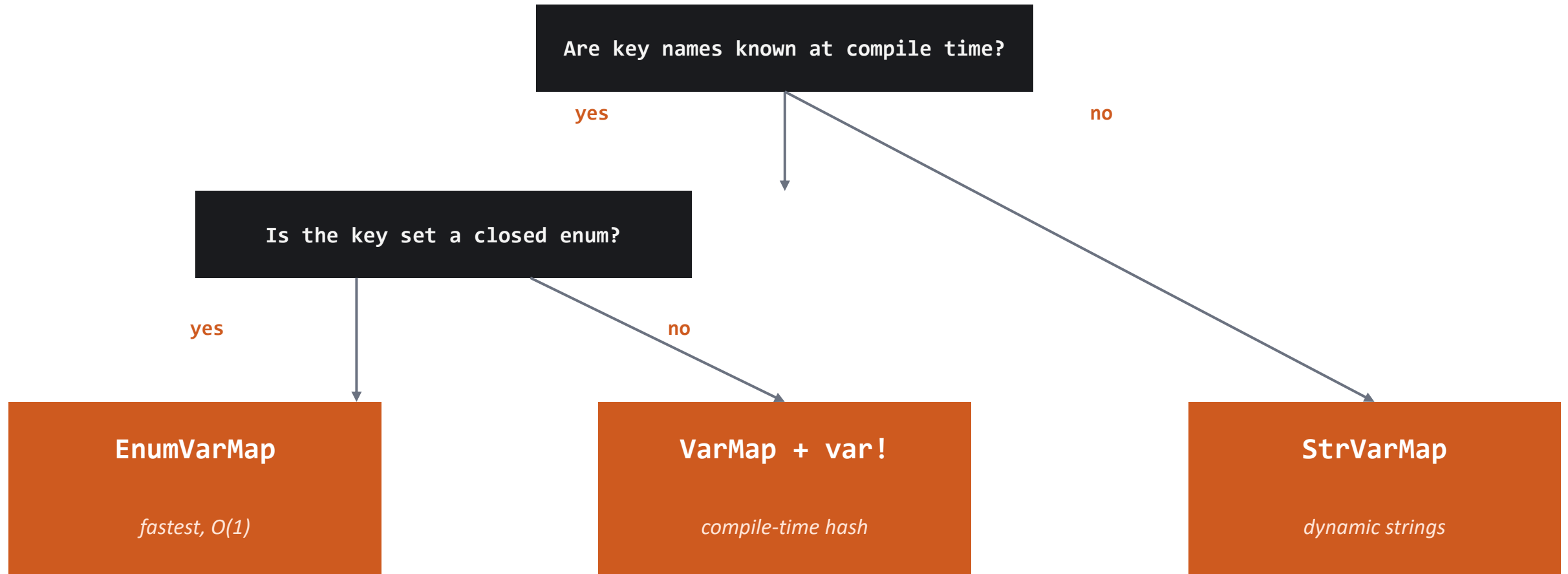


Update – small strings (ms, lower is better)



All three varmap types beat HashMap and BTreeMap, even on writes.

Choosing the right map



Tradeoffs

65 536-key ceiling

Internal index is 16-bit. Plenty for config / per-request state; not for unbounded sets.

Arena is append-only

Overwriting an arena-backed key (long string / bytes / custom) does not reclaim the old slot. Use `clear()` between snapshots.

Hash collisions are silent

VarMap uses FNV-1a; the top 48 bits identify the key. Statistically safe under 65k keys, but undetectable at runtime.

Custom types: Copy only

`#[derive(VarMapValue)]` requires Copy + alignment 1–16 + no generics. No Drop, no heap-owning fields.

GET STARTED

Try varmap today

```
[dependencies]  
varmap = "0.1"
```

C R A T E crates.io/crates/varmap

D O C S docs.rs/varmap

S O U R C E github.com/gdt050579/varmap

MIT-licensed · no_std-friendly value layer · 16-byte cells · arena-backed